Big Data Modeling & Analytics (course outline)

Mahmoud Parsian

Ph.D. in Computer Science

Big Data Modeling & Analytics?

1. What is a Big data Modeling?
2. What is a Big Data Modeling & Analytics?
3. Solutions to Big Data Modeling & Analytics

- MapReduce paradigm
- Spark/PySpark
- Amazon Athena & Google's Big Query

What is a Big Data Modeling?

Big data modeling is the process of organizing large, fast-moving, and diverse datasets so they can be stored, processed, analyzed, and understood efficiently.

Key Characteristics

1. Volume: Handles massive amounts of data
2. Velocity: Supports rapidly generated and processed data
3. Variety: Combines structured, semi-structured, and unstructured data
4. Veracity: The quality, reliability, and trust of the data
5. Scalability: Adapts as data and user demand grow

What is a Big Data Modeling?

Common Models

- Data lakes and Lake-Houses
- NoSQL models: document, key-value, column, and graph
- Distributed data warehouses (Snowflake)
- Batch and streaming architectures (Spark)

> Goal: Turn complex raw data into a reliable structure for analysis and decision-making.

What is a Big Data Modeling & Analytics?

Big data analytics uses modeled data to discover patterns,
explain outcomes, predict future events, and recommend actions.

Analytics Types

- Descriptive: What happened?
- Diagnostic: Why did it happen?
- Predictive: What is likely to happen?
- Prescriptive: What action should be taken?

Typical Workflow

Collect → Clean → Model → Process → Analyze → Visualize → Act

Business Value of Big Data Modeling & Analytics?

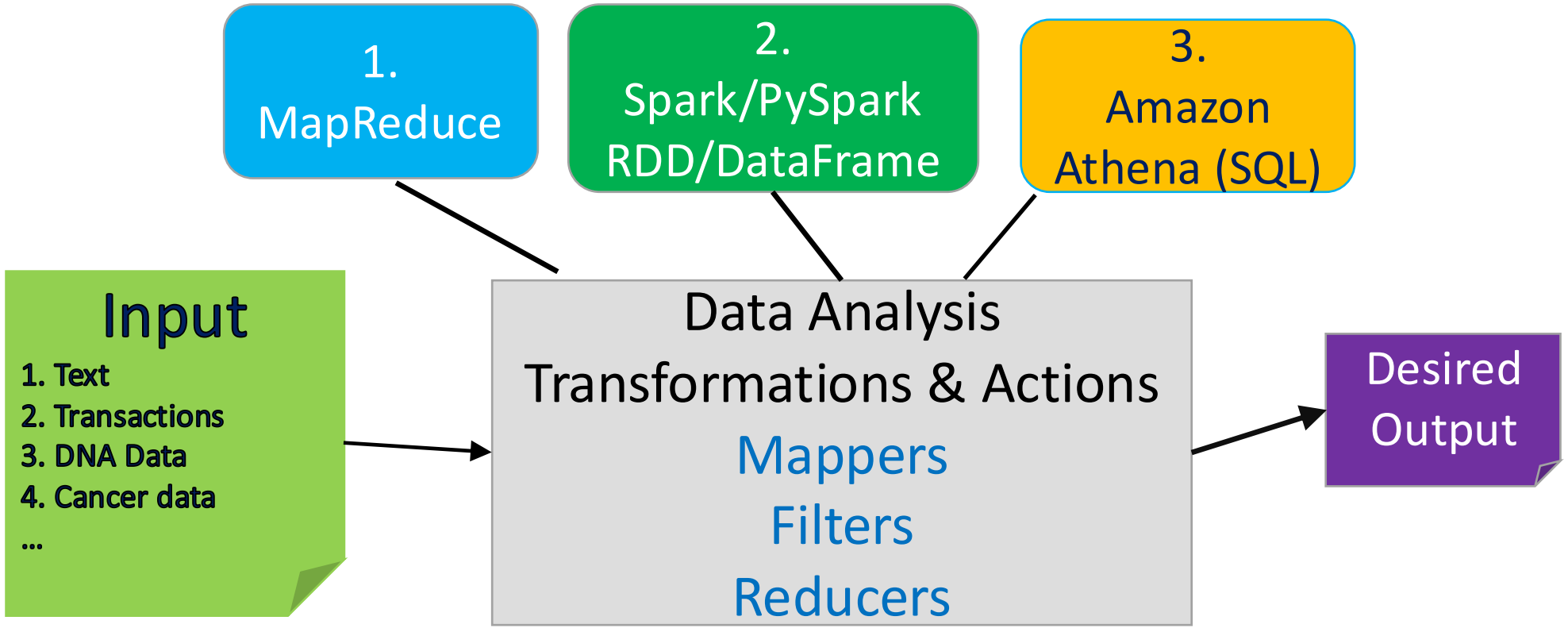
1. Faster, evidence-based decisions
2. Improved customer experiences
3. Fraud and risk detection
4. Operational efficiency
5. Forecasting and innovation
6. Building AI-LLM models

- Strong analytics depends on a scalable, accurate,
- and well-governed data model.

Main Course Components:

1. Introduction to MapReduce Paradigm (25%)
2. Spark and PySpark (60%)
3. Serverless SQL Access to Big Data (15%)

Big Data Analysis



1. MapReduce Paradigm

- **MapReduce** is a programming **paradigm/model** that enables **massive scalability** across hundreds or thousands of servers in a cluster.
- MapReduce has 3 functions:
  - `map()`
  - `reduce()`
  - `combine()`

1. MapReduce Programs

- **For MapReduce, we will use:**
  - Pseudo code (not executable)
  - PySpark code (runs on Spark)
- **For MapReduce, we will focus on learning MapReduce paradigm/model that enables massive scalability across hundreds or thousands of servers in a cluster.**
- **For MapReduce, pseudo-code**
  - examples are given in Lin, J., & Dyer, C. (2010). *Data-intensive text processing with MapReduce*

1. MapReduce Components

MapReduce has 3 functions:

map()

- Mapper function

reduce()

- Reducer function

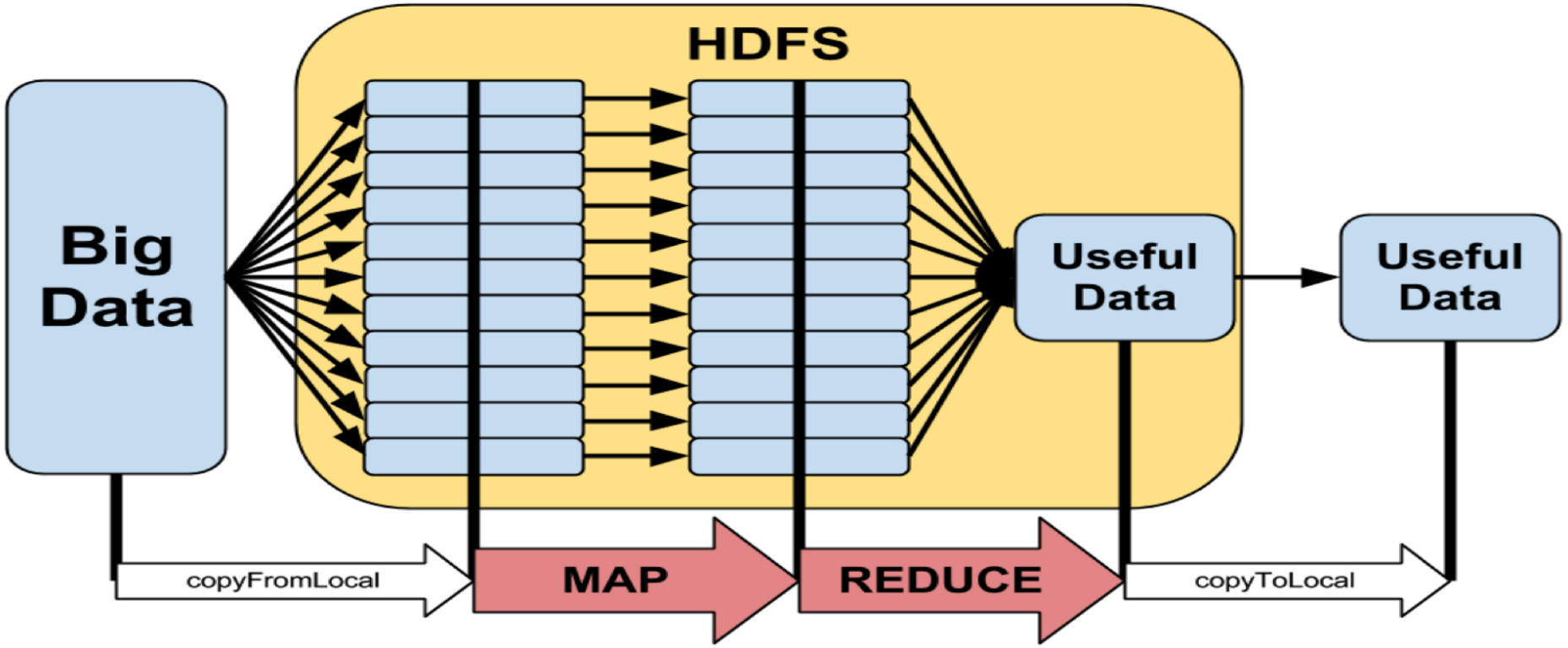
combine()

- Optional combiner function

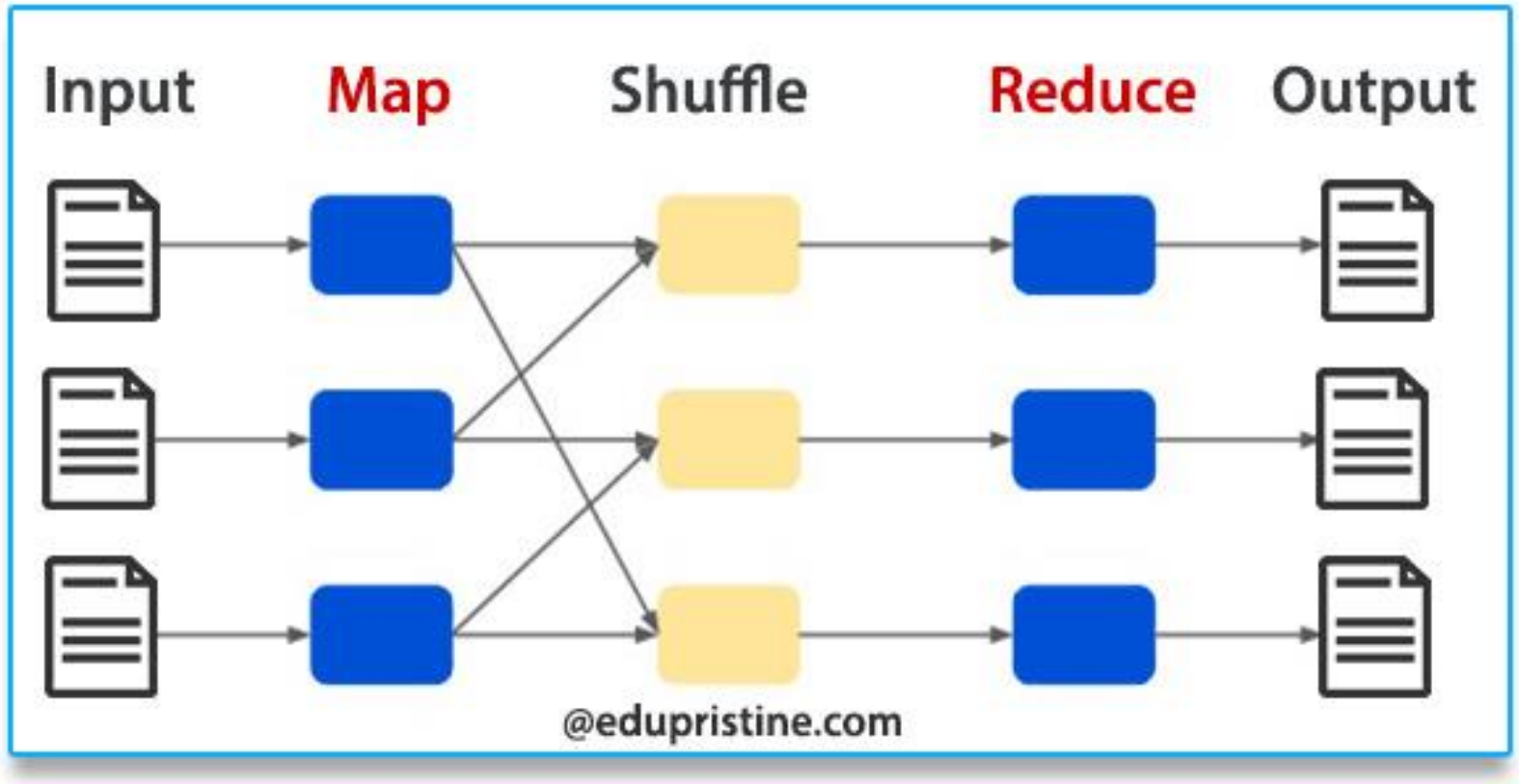
MapReduce Paradigm

- **MapReduce is a foundation model/paradigm for distributed computing using clusters**
- **MapReduce concrete implementations:**
  - Apache Hadoop implements MapReduce
  - Apache Spark implements superset of MapReduce
  - Apache Tez implements MapReduce
  - Amazon Athena implements MapReduce
  - Google Big Query implements MapReduce

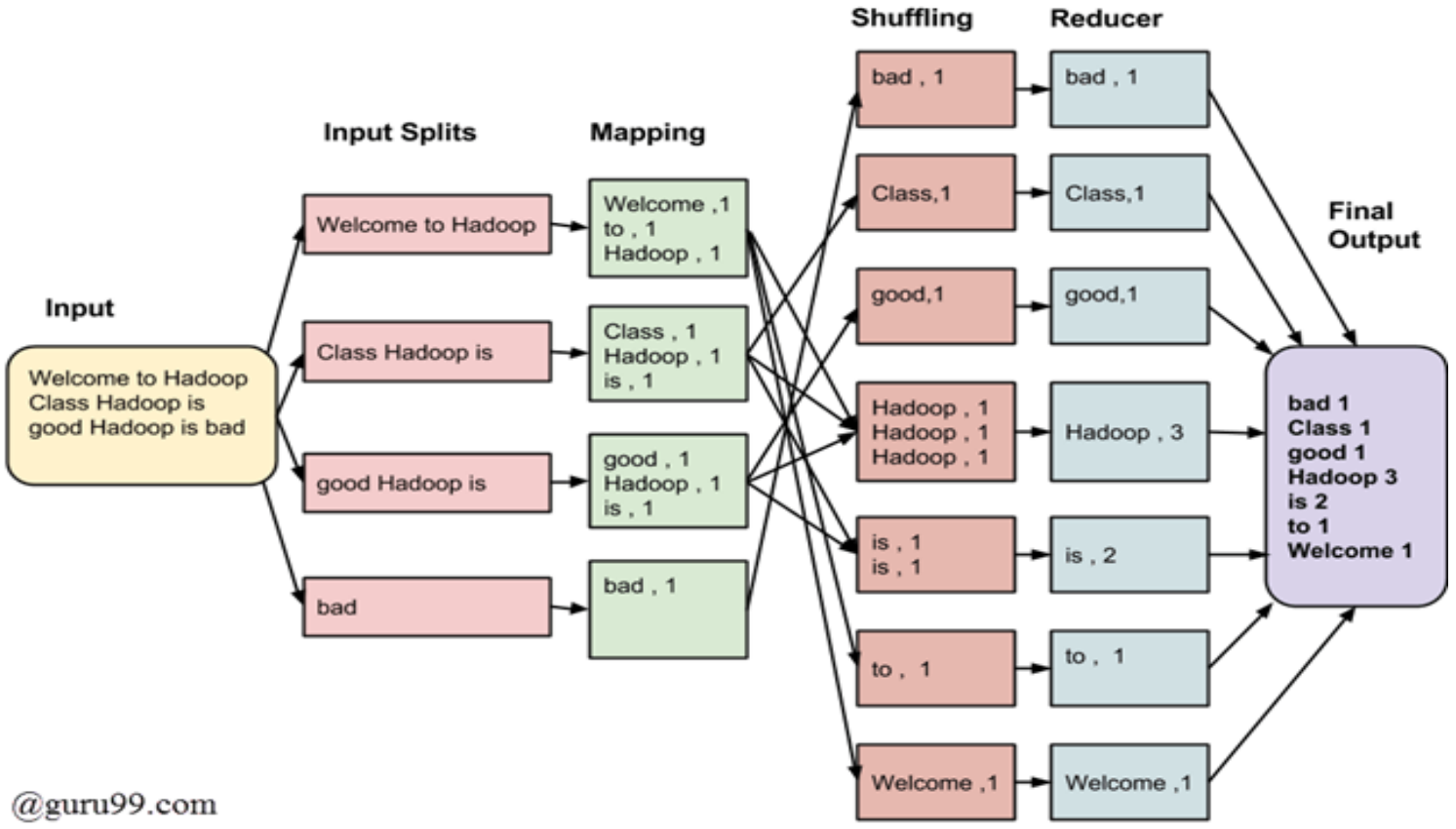
MapReduce Paradigm: High-Level



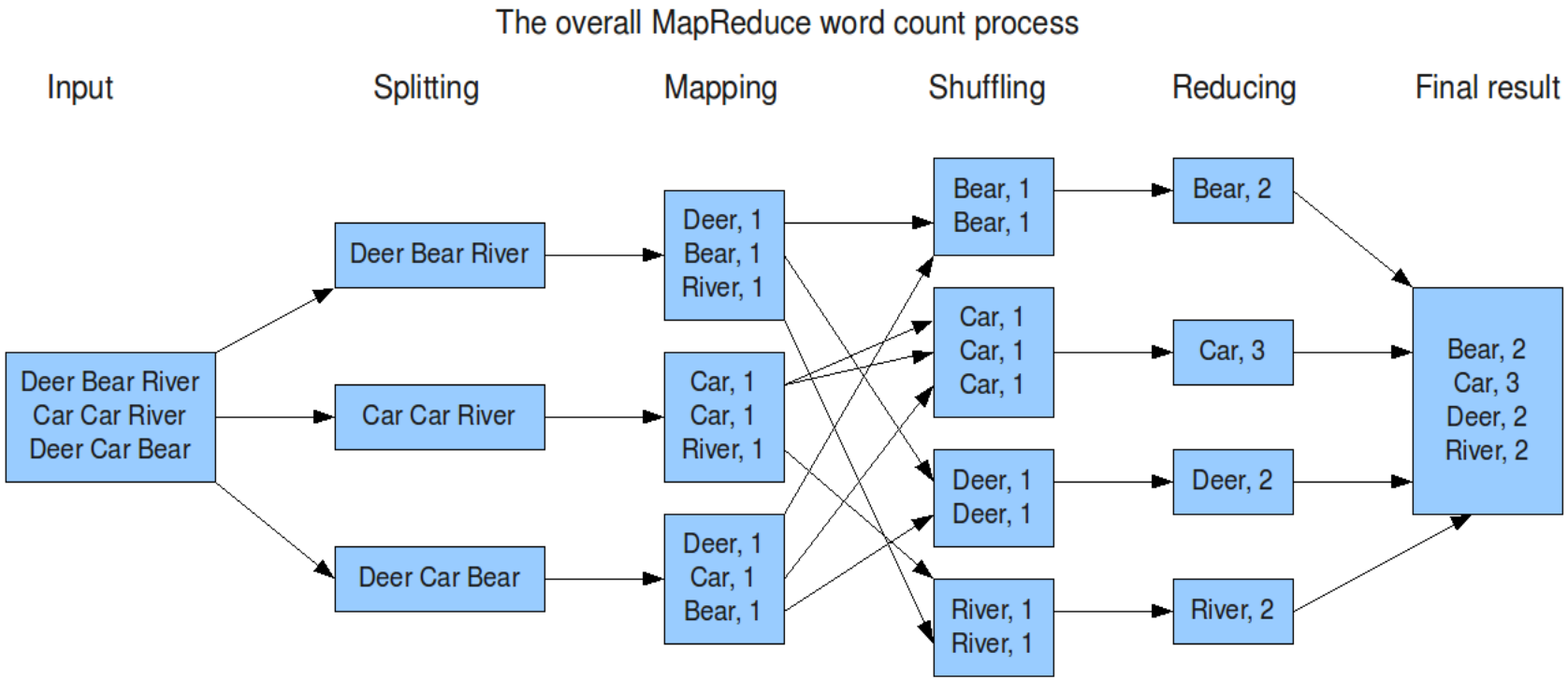
MapReduce Paradigm



MapReduce Paradigm Example-1



MapReduce Paradigm Example-2



Solving a Big Data Problem with MapReduce (1)

- **Input data is partitioned by a partitioner into chunks and sent to Mappers**
- **Assume your input is comprised of 80,000,000,000 records, which might be partitioned into 40,000 chunks**
  - Number of partitions: 40,000
  - Each partition will have about 2,000,000 records
  - Maximum mapper parallelism can be 40,000 mappers
  - 80,000,000,000 = 40,000 x 2,000,000
- **A mapper function, map(K, V) is executed on all partitions in parallel (as much as possible – depends on resources available).**
- **The map(K, V) generates any number of (key, value) pairs such as:**
- `{(K1, V1), (K2, V2), …}`**. For example, K can be a record number and V can be the actual input record**

Solving a Big Data Problem with MapReduce (2)

- All mappers output go to ***Sort & Shuffle*** phase, which groups values by unique keys (similar to GROUP BY in SQL).
- Sort & Shuffle creates (key, value) pairs as:

```
(key_1, [v1, v2, …])
(key_2, [u1, u2, …])
…
(key_n, [t1, t2, …])
```

Where `{key_1, key_2, … key_n }`

are the unique keys created by all mappers

Solving a Big Data Problem with MapReduce (3)

- Output of Sort & Shuffle goes to reducers
- A reducer operates on (key, value) pairs where key is in

```
{ key_1, key_2, …, key_n }
```

and value is an associated value for the key.

Therefore, the following reducers can be executed in parallel:

```
reduce(key_1, [v1, v2, …])
reduce(key_2, [u1, u2, …])
…
reduce(key_n, [t1, t2, …])
```

- **Each reducer can create any number of new (key, value) pairs.**

2. Apache Spark

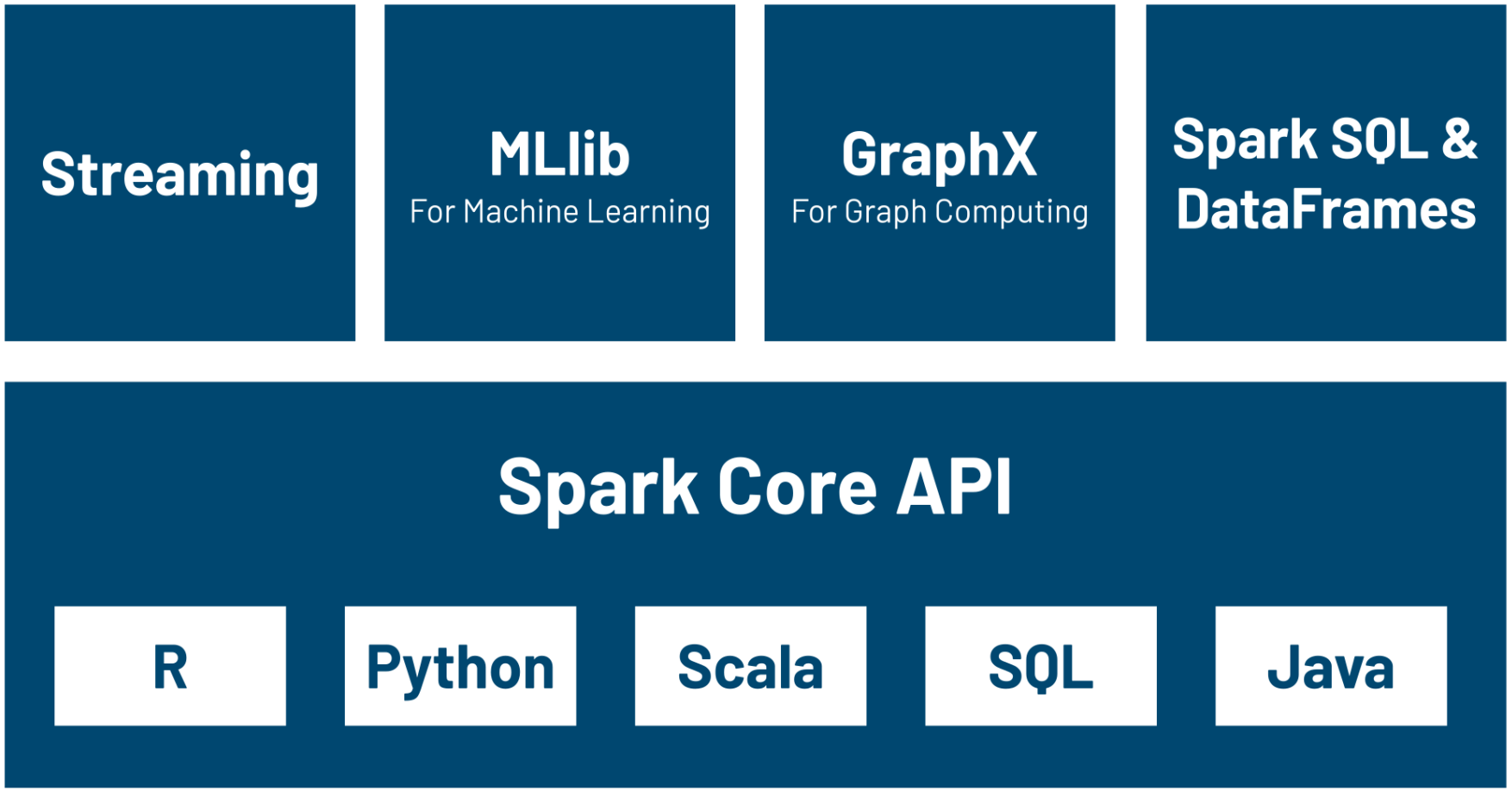
- Apache Spark is a multi-language (Java, Python, Scala) engine for executing data engineering, data science, and machine learning on single-node machines or clusters.
- PySpark is a Python API for Spark
- Spark is a superset of MapReduce
- Spark web site: https://spark.apache.org

2. Apache Spark Programs

For Apache **Spark** , we will use PySpark and write **actual executable programs:**

```
>>> # spark is an instance of SparkSession object
>>> rdd = spark.sparkContext
                    .parallelize(range(0, 100))
>>> rdd.count()
100
>>> sum_of_values = rdd.reduce(lambda x, y: x+y)
>>> sum_of_values
4950
```

Apache Spark: Components



Spark run modes

Local Mode Single machine:

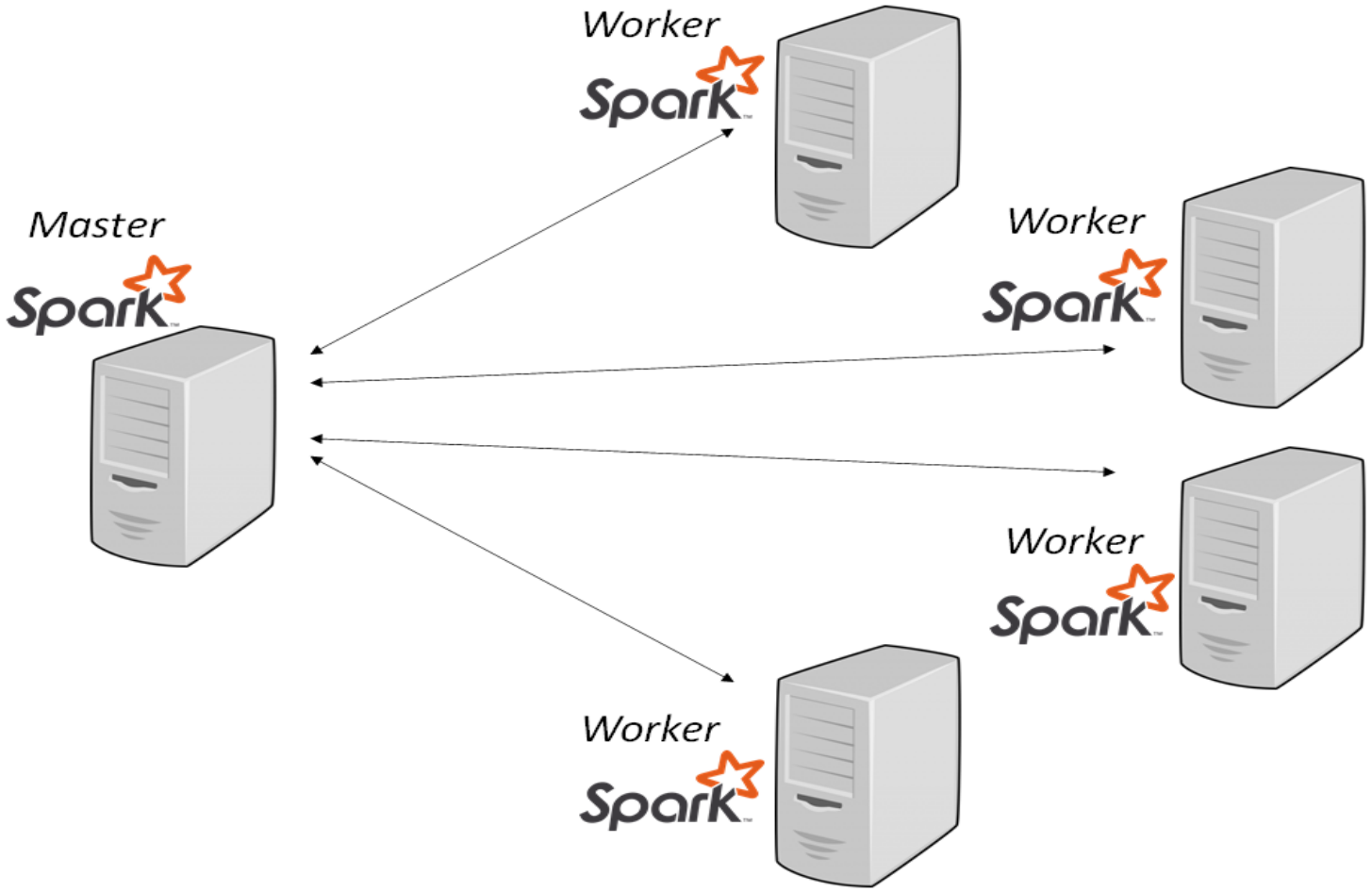
- Runs entirely on a single computer or laptop.
- Threads for execution: Uses local threads to simulate worker nodes.
- Testing and development: Best used for debugging and running quick tests.

Cluster Modes:

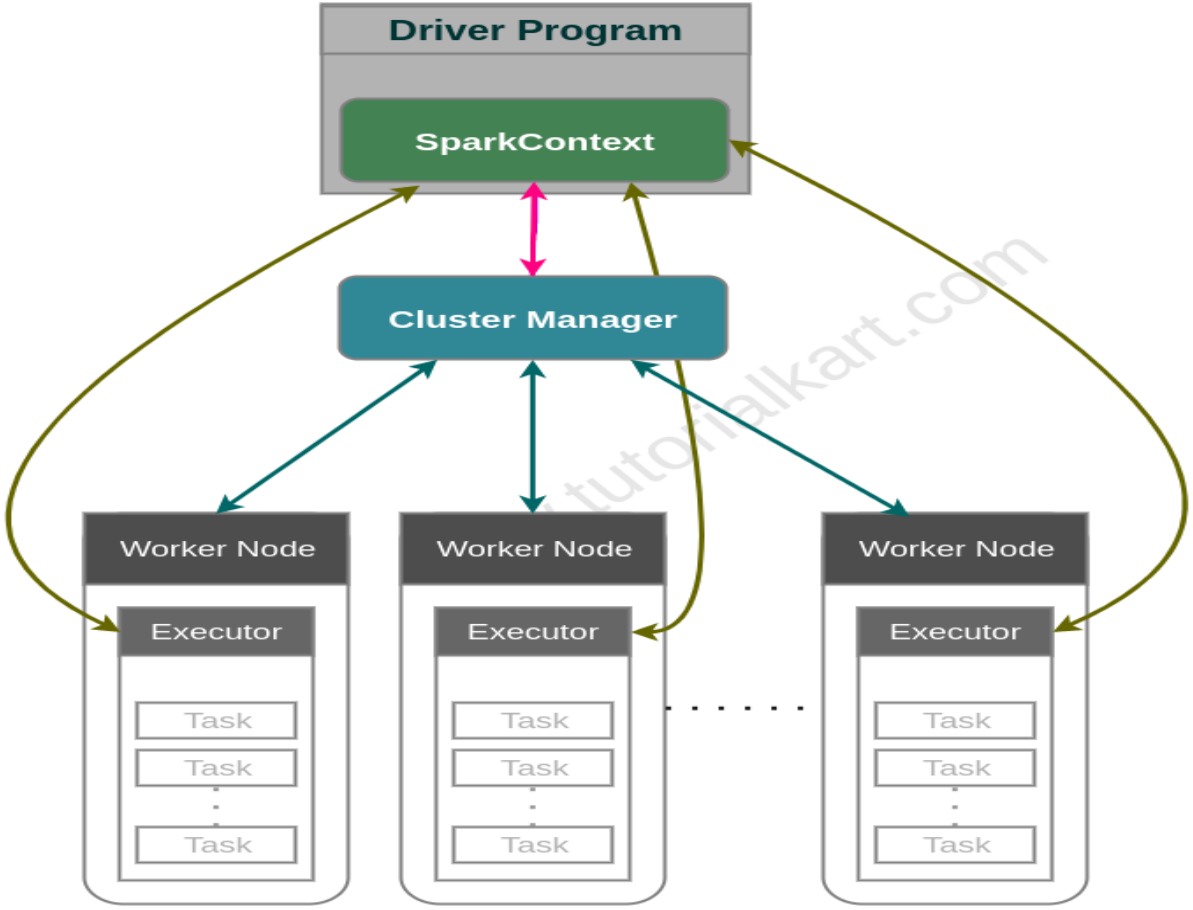
In cluster modes, the main distinction is where the Spark Driver (the program that coordinates the work) runs.

- **Client Mode**: External driver: The driver runs on the machine where the job was submitted. Interactive use: Best for interactive shells, notebooks, and local debugging. Network dependent: If your local machine disconnects, the job fails.
- **Cluster Mode**: Internal driver: The driver runs inside a worker node within the cluster. Production standard: Best for scheduled production jobs. Independent: You can close your laptop and the job will keep running.

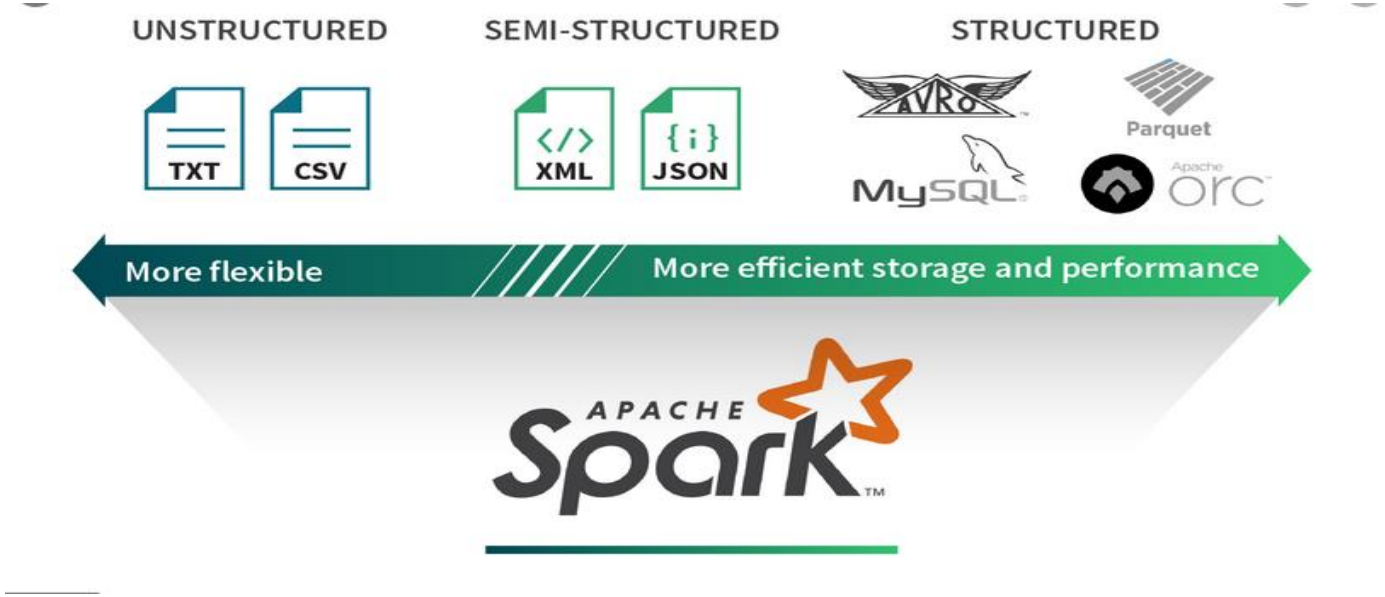
Apache Spark: runs in a cluster



Apache Spark: Cluster Manager



Spark Data Abstractions: Read/Write from/to Many Data Sources



Spark Data Abstractions

- **Your data can be represented as an RDD and DataFrame**
- **1. Resilient Distributed Datasets (RDD)**
  - Data is represented as a data type T
    (integer, string, tuples, arrays, dictionary, …)
  - Billions of data points (elements/records)
- **2. DataFrame**
  - Data is represented as a table of rows and named columns
  - Billions of rows of data

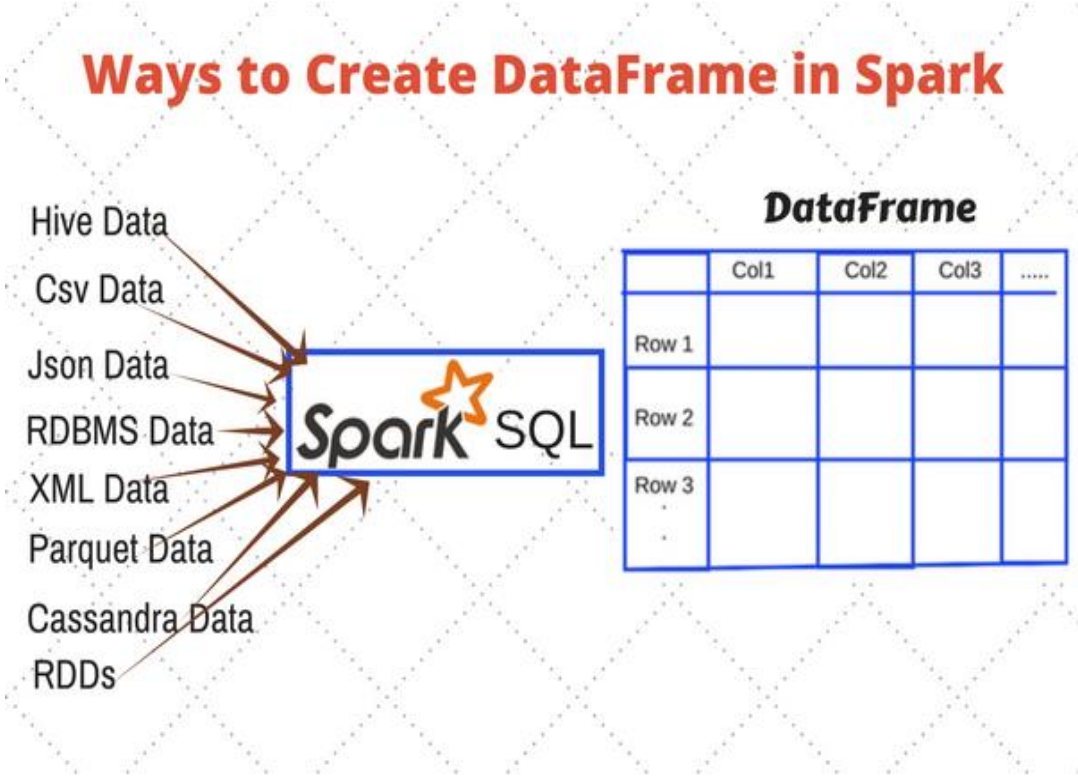
Spark Data Abstractions: RDDs

Billions of data elements

Each element: `(String, (Float, Float))`

Element-1:	`(gene-1, (3.0, 4.5))`
Element-2:	`(gene-2, (1.0, 1.5))`
Element-3:	`(gene-1, (2.1, 1.6))`
	…
	`(gene-8, (3.1, 5.1))`

Spark Data Abstractions: DataFrame



PySpark DataFrame Example

```
from pyspark.sql import SparkSession
# 1. Initialize a SparkSession
spark = SparkSession.builder.appName(”demo").getOrCreate()

# 2. Create a sample DataFrame
data = [ ("Alice", "Sales", 45000),
         ("Bob", "Engineering", 60000), …]
columns = ["name", "department", "salary"]
df = spark.createDataFrame(data, columns)

# 3. Register the DataFrame as a Temporary SQL View
df.createOrReplaceTempView("employees")
```

PySpark DataFrame Example continued...

```
# 4. Run an SQL query using spark.sql()
query_result = spark.sql("""
    SELECT
        department,
        COUNT(*) as total_employees,
        AVG(salary) as average_salary313131
    FROM employees
    GROUP BY department
    HAVING average_salary > 42000
    ORDER BY average_salary DESC
""")

# 5. Display the results
query_result.show()
```

3. Main Course Components
Serverless SQL Access to Big Data (15%)

- Amazon Athena
- Google BigQuery
- Snowflake

Amazon Athena

- Interactive query service
- Serverless. Zero infrastructure. Zero administration.
- Put your data in S3
- Access your data by SQL
- Pay by query
- Fast performance

Amazon Athena